

Statistical Learning Method

Hastie, T., Tibshirani, R., and Friedman, J. (2009). The Elements of Statistical Learning: Data Mining, Inference, and Prediction, Second Edition, Springer, New York.

<https://esl.hohoweiya.xyz/>

[统计学习基础 \(译注\) | 王冠嵩](#)

Introduction to Statistical Learning

Curse of Dimensionality

The **curse of dimensionality** is a fundamental issue in statistical learning and machine learning that arises when dealing with high-dimensional data. It refers to various problems that emerge as the dimensionality of data increases, making tasks such as distance computation, data visualization, and model generalization more difficult. Main issues including following three aspects.

1. Data Sparsity:

- In high-dimensional spaces, data points tend to be far apart from each other.
- Distance-based algorithms (e.g., KNN, clustering) become ineffective since all points appear almost equally distant.

2. Increased Computational Complexity:

- Many algorithms rely on pairwise distances or density estimation, which become computationally expensive as dimensions increase.

3. Overfitting in Machine Learning Models:

- With high dimensions, models become overly complex and capture noise rather than the true underlying pattern.
- More features require exponentially more training samples to achieve the same performance.

4. Reduced Discriminative Power of Distance Metrics:

- In high dimensions, Euclidean distance becomes less informative, as all points tend to have similar distances from each other (this is known as **distance concentration**).

Structured Regression Models

Consider the RSS criterion for an arbitrary function f ,

$$\text{RSS}(f) = \sum_{i=1}^N (y_i - f(x_i))^2.$$

Minimizing it leads to infinitely many solutions. In order to define the local behavior, constraints are imposed by restriction. Common methods for constraining include:

Classes of Restricted Estimators

Roughness Penalty and Bayesian Methods.

Here the class of functions is controlled by penalizing terms in forms of

$$\text{PRSS}(f; \lambda) = \text{RSS}(f) + \lambda J(f)$$

By selecting different $J(f)$, we can control the behavior of solutions. For instance, cubic smoothing spline introduces roughness penalty

$$\text{PRSS}(f; \lambda) = \sum_{i=1}^N (y_i - f(x_i))^2 + \lambda \int [f''(x)]^2 dx$$

which requires a small second derivative. In fact, penalty function, or **regularization** methods can be cast in a Bayesian framework. The penalty J corresponds to a log-prior, and $\text{PRSS}(f; \lambda)$ the log-posterior distribution, and minimizing $\text{PRSS}(f; \lambda)$ amounts to finding the posterior mode.

Kernel Methods and Local Regression

These methods can be thought of as explicitly providing estimates of the regression function or conditional expectation by specifying the nature of the local neighborhood, and of the class of regular functions fitted locally. The simplest form of kernel estimation is the Nadaraya–Watson weighted average:

$$\hat{f}(x_0) = \frac{\sum_{i=1}^N K_\lambda(x_0, x_i) y_i}{\sum_{i=1}^N K_\lambda(x_0, x_i)}.$$

In general we can define a local regression estimate of $f(x_0)$ as $f_{\hat{\theta}}(x_0)$, where $\hat{\theta}$ minimizes

$$\text{RSS}(f_\theta, x_0) = \sum_{i=1}^N K_\lambda(x_0, x_i) (y_i - f_\theta(x_i))^2,$$

and f_θ is some parameterized function, such as a low-order polynomial.

Basis Function

The **Basis Function Method** is a class of **restricted estimators** used in regression and function approximation. Instead of estimating a function directly, we approximate it using a **linear combination of basis functions**, which serve as building blocks to capture complex relationships in the data. Let $f(x)$ be the unknown function we want to estimate based on observed data $\{x_i, y_i\}$. Instead of modeling $f(x)$ directly, we approximate it as a weighted sum

of basis functions $\phi_j(x)$:

$$\hat{f}(x) = \sum_{j=1}^M w_j \phi_j(x)$$

Bias-Variance Trade-off and Gauss-Markov Theorem

Bias-Variance Trade-Off

The expected prediction error at x_0 , also known as test or generalization error, can be decomposed:

$$\begin{aligned} \text{EPE}_k(x_0) &= \mathbb{E}[(Y - \hat{f}_k(x_0))^2 | X = x_0] \\ &= \sigma^2 + [\text{Bias}^2(\hat{f}_k(x_0)) + \text{Var}_{\mathcal{T}}(\hat{f}_k(x_0))] \\ &= \sigma^2 + \left[f(x_0) - \frac{1}{k} \sum_{\ell=1}^k f(x_{(\ell)}) \right]^2 + \frac{\sigma^2}{k}. \end{aligned}$$

where the first term is irreducible, but later two terms are under our control. High bias and low variance is underfitting, and low bias with high variance means overfitting.

Gauss-Markov Theorem

The Gauss-Markov Theorem states that in a linear regression model, under certain assumptions, the **Ordinary Least Squares (OLS) estimator** is the Best Linear Unbiased Estimator. However, we can use biased estimator (e.g., shrinkage methods) to trade much smaller variance to minimize RSS.

EM Algorithm

The **Expectation-Maximization (EM)** algorithm is an iterative optimization technique used for **finding maximum likelihood estimates** of parameters in **probabilistic models with latent variables** (i.e., hidden or missing data).

Many real-world problems involve **incomplete data** or **latent (hidden) variables**, making it hard to directly optimize the likelihood. The EM algorithm offers a way to estimate parameters in such settings by iteratively applying two steps:

- **E-Step (Expectation)**: Estimate the missing data given current parameters.
- **M-Step (Maximization)**: Maximize the likelihood using the completed data from the E-step.

We want to maximize the log-likelihood:

$$\log p(X | \theta) = \log \sum_Z p(X, Z | \theta)$$

This is hard because of the summation over the latent variable Z . EM solves this by maximizing a **lower bound** instead:

$$\log p(X | \theta) \geq \mathbb{E}_{Z \sim q(Z)} \left[\log \frac{p(X, Z | \theta)}{q(Z)} \right]$$

EM Algorithm Steps

Given data X and latent variables Z :

1. **Initialize** model parameters $\theta^{(0)}$
2. **E-Step**: Compute the expected value of the complete data log-likelihood:

$$Q(\theta | \theta^{(t)}) = \mathbb{E}_{Z|X, \theta^{(t)}} [\log p(X, Z | \theta)]$$

This means: compute the **posterior probabilities** of the latent variables given the current parameters.

3. **M-Step**: Maximize the expected complete log-likelihood:

$$\theta^{(t+1)} = \arg \max_{\theta} Q(\theta | \theta^{(t)})$$

Supervised Learning

Decision Tree (CART)

Intuition and Mathematical Understanding

CART (Classification and Regression Trees) builds a binary tree by recursively splitting the data based on feature thresholds to maximize purity in the resulting subsets.

- For **classification**, it uses **Gini impurity** or **entropy** to measure how mixed the classes are.
- For **regression**, it minimizes **Mean Squared Error (MSE)** in the leaf nodes.
The model chooses the feature and split point that leads to the largest reduction in impurity (or error) at each step.

Strengths

- **Interpretability**: Easy to visualize and understand; decisions are made via simple rules.
- **No need for feature scaling**: Handles both numerical and categorical data directly.
- **Captures non-linear relationships**: Can model complex interactions between features.

Weaknesses

- **High variance**: Easy to overfitting, especially with deep trees.

- **Unstable:** Small changes in the data can lead to very different trees.
- **Biased splits:** May prefer variables with more levels (for classification).

Random Forest

Intuition and Mathematical Understanding

Random Forest is an **ensemble** of decision trees. It builds multiple trees on **bootstrapped** samples of the data and uses **feature randomness** (selects a random subset of features at each split) to reduce correlation between trees.

- **Classification:** Predicts the majority vote from all trees.
 - **Regression:** Averages the predictions from all trees.
- This reduces variance compared to a single decision tree.

Strengths

- **High accuracy:** Often performs well out-of-the-box.
- **Robust to overfitting:** Averaging reduces model variance.
- **Handles large feature spaces:** Works well even when many features are irrelevant.
- **Implicit feature selection:** Gives feature importance scores.

Weaknesses

- **Less interpretable:** Hard to visualize or explain compared to a single decision tree.
- **Computationally expensive:** Requires more memory and training time.
- **Not good at extrapolation:** Like all tree-based methods, it struggles outside the training data range.

Gradient Boosting Decision Tree (GBDT)

Intuition and Mathematical Understanding

GBDT is an **ensemble learning** technique that builds trees **sequentially**, each correcting the errors of the previous ones. At each step, it fits a new decision tree to the **negative gradient** (residuals) of the loss function.

Mathematically:

- We model the prediction as a sum of weak learners:

$$F_m(x) = F_{m-1}(x) + \gamma_m h_m(x)$$

- $h_m(x)$ is the new decision tree fitted to the residuals.
- γ_m is a learning rate to control step size.

Strengths

- **High predictive accuracy:** Often ranks top in ML competitions.
- **Flexible loss functions:** Can be used for classification, regression, ranking, etc.
- **Captures complex patterns:** Learns residuals stage-by-stage.

Weaknesses

- **Training is sequential:** Slower to train compared to parallelizable methods.
- **Sensitive to hyperparameters:** Requires careful tuning.
- **Overfitting risk:** Especially with deep trees or large learning rates.

XGBoost

Intuition and Improvements

XGBoost is an optimized implementation of GBDT with key enhancements:

- **Regularization:** Adds L_1/L_2 penalties to control overfitting.
- **Second-order approximation:** Uses both gradient and hessian for better optimization.
- **Parallelization:** Optimizes tree construction for speed.
- **Missing value handling:** Learns default directions for missing data.

Strengths

- **Fast and accurate:** Much faster than vanilla GBDT.
- **Built-in regularization:** Improves generalization.
- **Scalable:** Supports out-of-core learning and distributed computing.

Weaknesses

- **Complexity:** More difficult to tune than Random Forest.
- **Longer training time:** Still slower than LightGBM

LightGBM

Intuition and Improvements

LightGBM introduces two major innovations:

- **Leaf-wise growth:** Grows the tree by splitting the leaf with the highest loss reduction, not level-by-level.
- **Histogram-based binning:** Buckets continuous features into discrete bins to speed up training.

Strengths

- **Very fast:** Highly efficient for large datasets.
- **Low memory usage:** Due to histogram binning.
- **Better accuracy:** Leaf-wise growth often improves performance.

Weaknesses

- **Overfitting risk:** Leaf-wise growth can lead to deeper, more complex trees.
- **Not ideal for small data:** Needs enough data to benefit from histogram binning.

CatBoost

Intuition and Improvements

CatBoost is tailored for datasets with **categorical features**. Key features include:

- **Ordered boosting:** Avoids overfitting by using a special permutation technique to simulate online learning.
- **Efficient handling of categorical variables:** Converts categories into numbers using target statistics, carefully avoiding data leakage.

Strengths

- **Excellent for categorical data:** No need for one-hot encoding.
- **Stable and accurate:** Reduces overfitting via ordered boosting.
- **Minimal preprocessing:** Works well out of the box.

Weaknesses

- **Training time:** Slower than LightGBM on numerical-only data.
- **Documentation less mature:** Compared to XGBoost/LightGBM (though improving).

Support Vector Machine (SVM)

Intuition and Mathematical Understanding

SVM aims to find the **optimal hyperplane** that separates classes with the **maximum margin**. For non-linearly separable data, it uses the **kernel trick** to project data into a higher-dimensional space.

Mathematically, SVM solves:

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 \quad \text{subject to} \quad y_i(\mathbf{w} \cdot \mathbf{x}_i + b) \geq 1$$

With slack variables and kernels added for soft-margin and non-linear classification.

Strengths

- **Effective in high-dimensional spaces:** Especially with sparse features.
- **Robust to overfitting:** Especially when margin is large.
- **Flexible kernels:** Linear, polynomial, RBF, etc.

Weaknesses

- **Slow training:** Especially on large datasets.
- **Sensitive to parameter tuning:** C , kernel type, and γ need careful choice.
- **Hard to interpret:** Especially with complex kernels.

k-Nearest Neighbors (KNN)

Intuition and Mathematical Understanding

KNN is a **lazy, instance-based** learner. It makes predictions by looking at the **k closest training points** in feature space.

- **Classification:** Majority vote among the k nearest neighbors.
- **Regression:** Average of k neighbors' values.
Distance is typically computed using **Euclidean distance**, but others can be used.

Strengths

- **Simple and intuitive:** Easy to implement and understand.
- **No training phase:** All computation is deferred to prediction time.
- **Flexible:** Adapts to multi-class, regression, etc.

Weaknesses

- **Slow prediction:** High computational cost at inference.

- **Sensitive to feature scaling:** Distance-based.
- **Curse of dimensionality:** Distance becomes less meaningful in high dimensions.

Naive Bayes

Intuition and Mathematical Understanding

Naive Bayes is a **probabilistic classifier** based on **Bayes' Theorem**, assuming **conditional independence** between features given the class:

$$P(y | \mathbf{x}) \propto P(y) \prod_{i=1}^n P(x_i | y)$$

Different versions exist:

- **Gaussian NB:** For continuous features.
- **Multinomial NB:** For count data (e.g., text).
- **Bernoulli NB:** For binary features.

Strengths

- **Fast and scalable:** Very efficient in both training and inference.
- **Works well with high-dimensional sparse data:** Like text classification.
- **Robust to irrelevant features:** Due to independence assumption.

Weaknesses

- **Strong independence assumption:** Rarely holds in real-world data.
- **Poor with correlated features:** Can lead to inaccurate probabilities.
- **Not flexible:** Assumes fixed distribution form (e.g., Gaussian).

Unsupervised Learning

K-means Clustering

Intuition and Mathematical Understanding

K-means partitions data into **K clusters** by minimizing the **within-cluster sum of squares**. It iteratively:

1. Assigns each point to the nearest cluster center (centroid).
2. Updates the centroids to the mean of assigned points.

Objective function:

$$\min \sum_{i=1}^K \sum_{x \in C_i} \|x - \mu_i\|^2$$

Strengths

- **Simple and fast:** Efficient even for large datasets.
- **Easy to interpret:** Especially in low dimensions.
- **Scales well:** Time complexity is linear in number of samples.

Weaknesses

- **Need to predefine K:** Not always obvious.
- **Sensitive to initialization:** Poor seeds → poor clustering.
- **Only captures spherical clusters:** Assumes equal variance.
- **Not robust to outliers:** Can skew centroids.

Hierarchical Clustering

Intuition and Mathematical Understanding

Builds a tree of clusters (**dendrogram**) either:

- **Agglomerative** (bottom-up): Start with each point as its own cluster, and merge the closest ones iteratively.
- **Divisive** (top-down): Start with all points in one cluster, and recursively split.

Distances between clusters can be computed by:

- **Single linkage:** Closest pair.
- **Complete linkage:** Farthest pair.
- **Average linkage:** Mean of all pairwise distances.

Strengths

- **No need to specify K:** Dendrogram helps visualize cluster structure.
- **Captures nested clusters:** Good for hierarchical relationships.
- **Flexible distance metrics:** Can adapt to different data types.

Weaknesses

- **Not scalable:** $O(n^3)$ time complexity for naive implementation.
- **Sensitive to noise and outliers:** Especially with single linkage.
- **No reassignments:** Once merged/split, clusters are fixed.

DBSCAN (Density-Based Spatial Clustering of Applications with Noise)

Intuition and Mathematical Understanding

DBSCAN groups together points that are **densely packed**, and separates areas of low density as **outliers**.

- Points are labeled as:
 - **Core points:** Sufficiently many neighbors within ϵ -radius.
 - **Border points:** Fewer neighbors but within ϵ of a core point.
 - **Noise:** Neither core nor border.

Parameters:

- ϵ : Radius of neighborhood.
- **minPts:** Minimum number of neighbors to be a core point.

Strengths

- **Finds arbitrarily shaped clusters:** Unlike K-means.
- **No need to specify K:** Cluster count emerges from density.
- **Robust to outliers:** Identifies noise explicitly.

Weaknesses

- **Sensitive to ϵ and minPts:** Parameter tuning can be tricky.
- **Fails on varying density:** Struggles if clusters have different densities.
- **Not good in high dimensions:** Distance becomes less meaningful.

Gaussian Mixture Model (GMM)

Intuition and Mathematical Understanding

GMM models data as a **mixture of multiple Gaussian distributions**, assuming each cluster is Gaussian-shaped with its own mean and covariance. Unlike K-means, it gives **soft assignments** (probabilities) of points belonging to each cluster.

Mathematically:

$$p(x) = \sum_{k=1}^K \pi_k \mathcal{N}(x \mid \mu_k, \Sigma_k)$$

Parameters are estimated using the **Expectation-Maximization (EM)** algorithm:

- **E-step:** Estimate cluster membership probabilities.
- **M-step:** Update Gaussian parameters using those probabilities.

Strengths

- **Soft clustering:** Captures uncertainty in cluster assignment.
- **Flexible shapes:** Elliptical clusters via full covariance matrices.
- **Probabilistic foundation:** Good for statistical modeling.

Weaknesses

- **Assumes Gaussian distributions:** Not ideal for non-Gaussian clusters.
- **Sensitive to initialization:** May converge to local optima.
- **Requires K:** Like K-means, the number of clusters must be predefined.

t-SNE (t-distributed Stochastic Neighbor Embedding)

Intuition and Mathematical Understanding

t-SNE is a **non-linear dimensionality reduction** technique. It maps high-dimensional data to 2D/3D while **preserving local structures**.

Key idea:

- Convert pairwise distances in high and low dimensions to **probability distributions**.
 - Minimize the **Kullback-Leibler (KL) divergence** between these distributions.
- It uses a heavy-tailed **t-distribution** in low-dimensional space to avoid overcrowding.

Strengths

- **Excellent visualization:** Reveals local clusters and patterns.
- **Non-linear:** Captures complex relationships in data.

Weaknesses

- **No global structure:** Distances between clusters often meaningless.

- **Not scalable:** Slow for large datasets.
- **Not deterministic:** Results vary with each run unless seeded.
- **No inverse transform:** Can't map low $\rightarrow$ high dimensions.

UMAP (Uniform Manifold Approximation and Projection)

Intuition and Mathematical Understanding

UMAP is another **non-linear dimensionality reduction** method, grounded in **manifold learning** and **topology**.

Key idea:

- Construct a graph of nearest neighbors in high-dimensional space.
- Optimize a low-dimensional embedding to preserve this structure using **cross-entropy loss**.

UMAP tries to preserve both **local and some global structure**, more so than t-SNE.

Strengths

- **Faster than t-SNE:** Better scalability for large datasets.
- **Preserves more global structure:** Useful for downstream tasks.
- **Stable:** Less sensitive to randomness than t-SNE.
- **Can be used for supervised tasks:** Semi-supervised version exists.

Weaknesses

- **Still non-linear:** Interpretability of axes is low.
- **Parameter tuning:** `n_neighbors`, `min_dist` can change the outcome.
- **May distort fine details:** Especially with small `min_dist`.

Latent Dirichlet Allocation (LDA)

Intuition and Mathematical Understanding

LDA is a **generative probabilistic model** for **topic modeling**. It assumes:

- Each document is a mixture of topics.
- Each topic is a distribution over words.

It uses **Dirichlet priors** to model topic and word distributions.

Generative process:

1. For each document, sample a topic distribution $\theta \sim \text{Dir}(\alpha)$.
2. For each word:
 - Sample a topic $z \sim \text{Multinomial}(\theta)$
 - Sample a word $w \sim \text{Multinomial}(\beta_z)$

Strengths

- **Unsupervised:** Finds latent topics without labels.
- **Interpretable results:** Topics are human-readable word clusters.
- **Bayesian foundation:** Handles uncertainty in topic distribution.

Weaknesses

- **Bag-of-words assumption:** Ignores word order and syntax.
- **Sensitive to hyperparameters:** α and β tuning impacts results.
- **Not ideal for short texts:** Needs longer documents for meaningful topics.

Isolation Forest

Intuition and Mathematical Understanding

Isolation Forest is an **anomaly detection algorithm** based on the idea that **anomalies are easier to isolate**.

- Constructs random binary trees by recursively partitioning data.
- Anomalies require **fewer splits** to isolate.
- Anomaly score: Based on **average path length** over many trees.

Strengths

- **Efficient:** Linear time complexity with sub-sampling.
- **Works well in high dimensions:** Unlike distance-based methods.
- **Model-free:** No assumptions about data distribution.

Weaknesses

- **Randomness sensitivity:** Depends on number of trees and sampling.
- **Not good with categorical variables:** Needs encoding or transformation.
- **No explanation for anomalies:** Only scores are given, no feature-based reasoning.

Probabilistic Model

Bayesian Network (Bayes Net)

Intuition and Mathematical Understanding

A **Bayesian Network** is a **directed acyclic graph (DAG)** where:

- Nodes represent **random variables**.
- Edges represent **conditional dependencies**.
- Each node has a **conditional probability distribution (CPD)** given its parents.

Joint distribution is factorized as:

$$P(X_1, \dots, X_n) = \prod_{i=1}^n P(X_i \mid \text{Parents}(X_i))$$

Strengths

- **Interpretable structure**: Encodes causal or conditional relationships.
- **Modular**: Easy to update parts of the model.
- **Efficient inference**: When graph is sparse.

Weaknesses

- **Requires DAG**: No cycles allowed.
- **Structure learning is hard**: Especially with incomplete data.
- **Exact inference is NP-hard**: For complex graphs.

Hidden Markov Model (HMM)

Intuition and Mathematical Understanding

HMM is a **sequential probabilistic model** with:

- **Hidden states** (not observed).
- **Observed emissions** (dependent on hidden state).

Assumes:

- **Markov property**: Current state depends only on previous state.
- **Emission independence**: Observation depends only on current state.

Key components:

- Transition probabilities $P(z_t \mid z_{t-1})$
- Emission probabilities $P(x_t \mid z_t)$
- Initial state distribution $P(z_1)$

Strengths

- **Good for sequence data:** E.g., speech, text, bioinformatics.
- **Efficient algorithms:** Forward-Backward, Viterbi, Baum-Welch (EM).

Weaknesses

- **Strong assumptions:** First-order Markov and emission independence may not hold.
- **Fixed number of states:** Needs to be predefined.
- **Poor with long dependencies:** Limited memory of past.

Markov Random Field (MRF)

Intuition and Mathematical Understanding

MRF is an **undirected graphical model** where:

- Nodes = variables, edges = symmetric dependencies.
- Assumes **local Markov property**: A variable is conditionally independent of all others given its neighbors.

Joint distribution factorizes over **cliques** (fully connected subsets):

$$P(X) = \frac{1}{Z} \prod_{C \in \mathcal{C}} \psi_C(X_C)$$

Strengths

- **Models symmetric relationships:** Useful for spatial/vision data.
- **Flexible:** No DAG constraints like Bayesian networks.
- **Rich structure:** Encodes arbitrary neighborhood systems.

Weaknesses

- **Inference is difficult:** Especially computing Z .
- **Parameter learning is hard:** Non-convex optimization.
- **Less intuitive:** Compared to Bayes nets for causal modeling.

Conditional Random Field (CRF)

Intuition and Mathematical Understanding

CRF is a type of **discriminative probabilistic graphical model** used primarily for **structured prediction** — predicting a sequence of outputs given a sequence of inputs (e.g., part-of-speech tagging, named entity recognition).

- Unlike HMM (generative), CRF directly model the **conditional probability**:

$$P(\mathbf{y} \mid \mathbf{x}) = \frac{1}{Z(\mathbf{x})} \exp \left(\sum_i \sum_k \lambda_k f_k(y_i, y_{i-1}, \mathbf{x}, i) \right)$$

- f_k : feature functions
- λ_k : learned weights
- $Z(\mathbf{x})$: normalization factor (partition function)

CRF can incorporate **arbitrary, overlapping, non-independent features** of the input — something HMM cannot do.

Strengths

- **Global sequence optimization**: Makes joint predictions over output labels, not independent decisions.
- **Flexible feature design**: Can use domain knowledge without making independence assumptions.
- **Good for NLP tasks**: Like sequence labeling, chunking, and information extraction.

Weaknesses

- **Training is computationally expensive**: Especially due to the partition function.
- **Requires labeled sequences**: Supervised learning only.
- **Limited scalability**: Hard to use with very large label spaces or long sequences.
- **Manual feature engineering**: Still often needed (unless combined with neural nets).

MCMC

Intuition and Mathematical Understanding

Monte Carlo methods are a class of algorithms that rely on **random sampling** to estimate numerical results, particularly for problems with complex integrals or high-dimensional spaces.

MCMC (Markov Chain Monte Carlo) is a family of Monte Carlo algorithms designed to **sample from a probability distribution** when direct sampling is difficult.

Key idea:

- Construct a **Markov chain** whose stationary distribution is the target distribution $P(x)P(x)P(x)$.
 - Run the chain long enough, and samples from the chain approximate $P(x)P(x)P(x)$.
- Common algorithms:

- **Metropolis-Hastings:**

- Propose a new state $x' \sim q(x' | x)$
- Accept with probability:

$$\alpha = \min \left(1, \frac{P(x')q(x | x')}{P(x)q(x' | x)} \right)$$

- **Gibbs Sampling:** Update each variable in turn by sampling from its **conditional distribution**.

Strengths

- **General-purpose inference:** Can approximate any distribution given enough time.
- **Essential for Bayesian inference:** Useful when posterior is analytically intractable.
- **Applicable in high dimensions:** Especially when factorized structure exists (e.g., in graphical models).

Weaknesses

- **Slow convergence:** Can require many samples to mix (burn-in period).
- **Autocorrelated samples:** Not all samples are independent; effective sample size is lower.
- **Hard to diagnose:** Requires careful tuning and convergence checking.
- **Computationally intensive:** Especially for complex or multimodal distributions.